

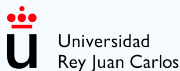
Big Data Processing II

Tema 2: Spark en Streaming

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Febrero 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Entender qué problema resuelve en el **análisis de datos deportivos**
- Conocer los conceptos clave de **Spark Structured Streaming**
- Aplicar **casos de uso** en datos deportivos en tiempo real
- Saber **optimizar consultas** y rendimiento en streaming



- 1 **Introducción**
- 2 **Componentes**
- 3 **Infraestructura**
- 4 **Ejemplos**



- 1 **Introducción**
- 2 Componentes
- 3 Infraestructura
- 4 Ejemplos



- **Procesamiento de datos en tiempo real**
 - Datos que llegan continuamente (logs, sensores, eventos)
 - Procesamiento incremental a medida que llegan
 - Latencia baja (segundos) vs batch (horas/días)
- **Structured Streaming proporciona una API unificada**
 - Misma API que en batch (DataFrames/SQL)
 - Spark SQL se encarga de la gestión del streaming
 - Garantías de consistencia y tolerancia a fallos
- **Para monitorización en tiempo real, alertas, ETL continuo**



- **Spark DStream (legacy)**
 - Basado en RDDs y micro-batching rígido
 - Opera a nivel de tiempo de proceso
 - API de más bajo nivel
- **Structured Streaming (recomendado)**
 - Basado en DataFrames/Datasets
 - Soporte nativo en tiempo de evento
 - Optimizado por Catalyst Optimizer
- **Se trabajará con Spark Structured Streaming**



- **Structured Streaming tiene dos modos**
- **Micro-batch (por defecto)**
 - Ejecuta el stream como una secuencia de micro-batches
 - Compatibilidad con la mayoría de operadores y destinos
 - Latencia de ~ 100 ms (mínimo) a segundos
- **Continuous processing (experimental)**
 - Pensado para latencias muy bajas
 - Menos operadores soportados y más restricciones
 - Latencia de ~ 1 ms



- **Un stream es una tabla que crece continuamente**
 - Cada evento nuevo es una fila que se añade
 - Una consulta genera resultados de forma incremental
- **Un flujo continuo de eventos es una tabla infinita**

Modelo tabular de streaming

Esquema conceptual original de la edición abierta



- **Se programa como en procesamiento en batch**
 - API SQL/DataFrame con transformaciones encadenadas
 - Misma lógica para datos finitos y para flujos continuos
- **Modelo mental: *tabla* → *query* → *salida***
 - El *stream* se interpreta como una tabla que crece
 - La query produce resultados de forma incremental
- **Enfoque declarativo**
 - Defines *qué* calcular, no los detalles de ejecución
 - Spark planifica y optimiza el *cómo*



- **Misma API que en batch**

- Operaciones habituales (`select`, `join`, `count`, ...)
- Permite reutilizar lógica batch en pipelines de streaming

- **Particularidades en streaming**

- Resultados incrementales → la salida se actualiza con nuevos datos
- Gestión de estado → agregaciones/ventanas
- Semánticas de salida → *append*, *update*, *complete*



- **Consulta declarativa sobre un stream de eventos**
 - Se expresa como una query de tipo batch (DataFrame/SQL)
 - Se evalúa incrementalmente en cada trigger con los datos nuevos
 - Spark SQL la planifica y optimiza (Catalyst + Tungsten)
- **Los resultados se escriben en un destino de salida**
- **Ejemplos son resultados que se actualizan en cada trigger**



Ejemplo de conteo continuo

Esquema conceptual original de la edición abierta



- 1 Introducción
- 2 Componentes**
- 3 Infraestructura
- 4 Ejemplos



Arquitectura de Structured Streaming

Esquema conceptual original de la edición abierta

Entrada → Transformaciones (sin/con estado) → Salida



Flujo de Structured Streaming

Esquema conceptual original de la edición abierta



- **SparkSession es el punto de entrada**
 - Las entradas (sources) se leen con `spark.readStream`
 - Las salidas (sinks) se escriben con `df.writeStream`
- **Del mismo stream se pueden ejecutar múltiples queries**
 - Query 1 → resultados por ventana (como posesión) para un dashboard
 - Query 2 → detecta eventos críticos (posible lesión) y los publica en Kafka



- **Lectura de streams con `spark.readStream`**
- **Soporta múltiples fuentes habituales**
 - Kafka como sistema de mensajería
 - File para nuevos ficheros en un directorio
 - Socket para pruebas rápidas (Netcat)
 - Rate para testing y benchmarking
- **El paralelismo suele venir de la fuente**
 - Como particiones de tópicos de Kafka
 - Más particiones $\Rightarrow$ más tareas en paralelo

Plataforma de eventos Kafka

Esquema conceptual original de la edición abierta



- **Control de ejecución de una query**

- `query = df.writeStream... .start()` inicia una query
- `query.awaitTermination()` bloquea hasta que termine
- `query.stop()` detiene la query (graceful stop)

- **Operación de queries en producción**

- Poner un nombre descriptivo a la query
- Un mismo job (programa) puede ejecutar múltiples queries en paralelo



- **En streaming el esquema es esencial**
 - En un stream infinito no se recomienda inferir
 - Es importante definir el esquema (JSON)
 - Esto incluye tipar los campos de eventos
- **Decisiones de parsing de eventos**
 - Registros corruptos → enviar a cola aparte o fallar
 - Evolución de esquema → versionado/validación



- **Operaciones sin recordar el pasado**

- Suelen escalar mejor → no requieren estado compartido entre micro-batches
- Algunas son `select`, `filter`, `withColumn`, `explode`, `map`, ...
- Filtrar `event_type == "SHOT"` o extraer minutos de un timestamp

- **Si no hay agrupación/ventanas/join temporal/dedup normalmente es stateless**



- **Matienen el estado entre micro-batches**
 - Recuerdan eventos anteriores
- **Tipos de operaciones con estado**
 - Agregaciones → `groupBy`, `count/sum/avg`
 - Deduplicación → `dropDuplicates`
 - Joins → `stream-static` y `stream-stream`
 - Estado custom → `mapGroupsWithState`



- **Propiedades a alto nivel**

- Estado actualizado incrementalmente entre micro-batches
- Recuperación y tolerancia a fallos mediante checkpointing
- Limpieza y estado acotado con watermarks o timeouts

- **Ejemplo → resultados por ventanas de últimos 5 minutos**



- **Stream-to-Static**

- Enriquecer un stream con datos (relativamente) estáticos
- Ejemplo → eventos del partido en streaming + datos de la plantilla

- **Stream-to-Stream**

- Correlacionar eventos entre dos streams
- Requiere tiempo de evento + watermarks + rango temporal para acotar estado
- Ejemplo → eventos del partido + biometría en streaming

- **Joins stream-stream sin cota temporal crecen el estado indefinidamente**



- **Eventos tardíos / fuera de orden**
 - Cuando ocurrió $\neq$ Cuando llegó
 - Ventanas/agregaciones usan tiempo de evento
 - El trigger marca cada cuánto se procesa
- **Watermark define la tardanza aceptada**
 - Cierra ventanas y acota estado
 - Define hasta cuándo se procesan eventos tardíos



- **Agrupaciones en intervalos de tiempo**
- **Ventanas fijas (tumbling)**
 - Bloques fijos y no superpuestos
- **Ventanas deslizantes (sliding)**
 - Bloques que se superponen
- **Ventanas de sesión (session)**
 - Separados por gap de inactividad
- **Es una columna más de agrupación**

Ventanas temporales

Esquema conceptual original de la edición abierta



Agregación por ventana

Esquema conceptual original de la edición abierta



- **Define cada cuánto se ejecuta la query para procesar nuevos datos**
 - Default → tan rápido como sea posible (micro-batch)
 - ProcessingTime → intervalo fijo (por ejemplo cada 10s)
 - AvailableNow → procesa lo disponible y se detiene
 - Continuous → sólo en modo continuous (con restricciones)



- **Define cuánto esperar un evento tardío**
 - Los eventos no siempre llegan en orden ni a tiempo
 - El GPS de un jugador puede llegar 2 minutos tarde al servidor
 - El watermark define cuánto retraso toleramos antes de cerrar una ventana
- **Su uso es fundamental para controlar el estado**
 - Un watermark de 2 minutos (tiempo de evento) acepta tardíos hasta 2 min
 - Más allá los tardíos no actualizan ventanas/estado
- **Supone un trade-off**
 - Watermark amplio → más precisión pero más memoria usada
 - Watermark corto → resultados más rápidos pero posibles eventos perdidos



- **Es habitual que lleguen eventos tardíos**

- Dentro del watermark → se incluye en el cálculo y puede corregir un resultado emitido
- Fuera del watermark → se descarta para mantener los resultados estables

- **Ejemplo → tiros a puerta en los últimos 5 minutos**

- Un sensor registra un tiro a las 18:32 pero el dato llega a las 18:34
- Con watermark de 3 minutos → se acepta y corrige del intervalo 18:30–18:35
- Con watermark de 1 minuto → llega demasiado tarde y se descarta



Gestión de datos tardíos

Esquema conceptual original de la edición abierta



- **Append escribe solo filas nuevas y finales**
 - No reescribe filas
 - En ventanas emite cuando la ventana se “cierra”
 - Útil para eventos crudos o resultados finales
- **Update escribe filas que cambian**
 - Reescribe solo las filas actualizadas desde el último trigger
 - Útil para resultados en vivo (contadores en evolución)
- **Complete reescribe la tabla completa en cada trigger**
 - Solo viable si la tabla de resultados es pequeña



Watermark en modo append

Esquema conceptual original de la edición abierta



Watermark en modo update

Esquema conceptual original de la edición abierta



- **Escritura de resultados con `writeStream`**
- **Soporta múltiples destinos habituales**
 - Sistemas de mensajería (Kafka)
 - Bases de datos SQL y NoSQL (InfluxDB)
 - Archivos como HDFS/S3 (Parquet/JSON)
 - Consola/memoria para depuración y desarrollo
 - `foreach/foreachBatch` para lógica personalizada

Visualización operativa

Esquema conceptual original de la edición abierta



- 1 Introducción
- 2 Componentes
- 3 Infraestructura**
- 4 Ejemplos



- **Eliminar duplicados en streaming**

- La fuente puede reemitir eventos (retries, reconexiones, *at-least-once*)
- Se deduplica por una clave estable (como `event_id`)
- Se usa watermark → limita cuánto tiempo recordamos



- **Las operaciones con estado consumen recursos**
 - Spark necesita recordar eventos pasados para calcular agregaciones y ventanas
 - Más claves de agrupación o ventanas más amplias $\Rightarrow$ más memoria necesaria
- **El watermark limita cuánto estado se acumula**
 - Permite a Spark olvidar ventanas ya cerradas
 - Sin watermark el estado crece indefinidamente
- **Regla práctica: agrupa por pocas claves y usa ventanas razonables**



- **El trigger controla cada cuánto se actualizan los resultados**
 - Trigger corto → datos más frescos pero más carga en el sistema
 - Trigger largo → datos menos frescos pero menos carga
- **Regla práctica: elige el trigger según la necesidad del análisis**
 - Dashboard en tiempo real → trigger de segundos
 - Reporte por minuto → trigger de 60s es suficiente



- **Spark recuerda hasta dónde llegó**
 - Guarda un punto de control tras cada micro-batch
 - Si el proceso cae, retoma desde ahí sin reprocesar datos ya calculados
 - Evita duplicados y pérdida de resultados intermedios
- **Checkpointing para recuperación tras fallos**
 - Persiste progreso y estado
 - Spark persiste logs de progreso/commits asociados a cada trigger
 - Tras un reinicio reanuda desde el último progreso consistente



- **Spark garantiza que cada evento se procesa exactamente una vez**
 - Los resultados no se duplican ni se pierden aunque el sistema falle y se reinicie
- **Para que funcione se necesitan tres condiciones**
 - La fuente permite releer datos (como Kafka)
 - Spark guarda el progreso (checkpointing)
 - El destino tolera escrituras duplicadas sin problemas



- 1 Introducción
- 2 Componentes
- 3 Infraestructura
- 4 Ejemplos**



- Se recomienda despliegue local / Docker para pruebas
- **Requisitos de instalación en despliegue local de Spark**
 - Java instalado (`java --version`) y configurar `JAVA_HOME`
 - Instalar PySpark y crear una sesión local con todos los núcleos [*]

Terminal Shell Terminal

```
java --version           # verificar Java
apt install -y openjdk-21-jdk  # instalar Java (Colab / Linux)
pip install pyspark       # instalar PySpark
```



- Si `JAVA_HOME` no está definida hay que indicarla manualmente

Python Python Code

```
import os
import pyspark

# En Debian/Ubuntu con version 21 de Java:
# os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-21-openjdk-amd64"
# En Windows con version 21 de Java:
# os.environ["JAVA_HOME"] = "C:\\Program Files\\Java\\jdk-21"

print(os.environ["JAVA_HOME"]) # comprobar si esta definida
print(pyspark.__version__)
```



- Jupyter + PySpark ya instalados y configurados

Terminal Shell Terminal

```
docker run \  
  --network=host \  
  -e JUPYTER_TOKEN="MY_SECRET" \  
  -v ${PWD}:/home/jovyan/work \  
  jupyter/pyspark-notebook
```



```
services:
  jupyter:
    image: jupyter/pyspark-notebook
    network_mode: host
    environment:
      - JUPYTER_TOKEN=MY_SECRET
    restart: on-failure
    volumes:
      - ./:/home/jovyan/work
```



❶ Leer líneas Spark desde un socket en el puerto 9999

- Cada línea representa un evento de un partido de fútbol
- Pueden ser goal, shot, foul y pass

❷ Filtrar solo eventos válidos y normalizar cada línea

❸ Agregar el conteo acumulado por tipo de evento

❹ Escribir los resultados en consola



- **Paso 0 - Simular un flujo de tipos de eventos de un partido de fútbol**
 - Socket con Netcat para enviar eventos de prueba
 - En Windows instalar Nmap (<https://nmap.org/download>)

Terminal Shell Terminal

En local

```
nc -lv 9999
```

En docker

```
docker run --rm -it --init -p 9999:9999 madrueno/nc -lv 9999
```



• Paso 1 - Iniciar una sesión de Spark Structured Streaming

Python Python Code

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, trim

spark = (
    SparkSession.builder                # Construye la sesión de Spark
    .appName("FootballStreamingEventCount") # Nombre descriptivo
    .getOrCreate()                      # Crea o reutiliza la sesión existente
)
```



- Paso 2 - Leer líneas de eventos desde el socket abierto

Python Python Code

```
lines = (  
    spark.readStream                # Lee un stream de una fuente  
    .format("socket")              # Fuente de tipo socket  
    .option("host", "localhost")   # Host del socket  
    .option("port", 9999)          # Puerto del socket  
    .load()                        # Carga como DataFrame  
)
```



• Paso 3 - Normalizar texto y filtrar eventos válidos

- Convertir a minúsculas y eliminar espacios
- Filtrar solo los tipos de eventos de interés

Python Python Code

```
valid = ["goal", "shot", "foul", "pass"]           # Tipos de eventos válidos

events = (
    lines                                           # DataFrame (string por defecto)
    .select(trim(lower(col("value"))).alias("event")) # Normaliza texto
    .where(col("event").isin(valid))               # Filtra solo eventos válidos
)
```



● Paso 4 - Agregar el conteo acumulado por tipo de evento

- Agrupa por tipo y cuenta el número de eventos por tipo
- La query no se ejecuta hasta que se escriba la salida

Python Python Code

```
counts = (  
    events                # DataFrame con eventos  
    .groupBy("event")     # Agrupa por tipo de evento  
    .count()              # Cuenta el número de eventos por tipo  
)
```



- Paso 5 - Escribir resultados continuamente en consola

Python Python Code

```
query = (  
    counts.writeStream          # Define el destino de salida  
    .outputMode("complete")    # Muestra la tabla completa de conteos  
    .format("console")         # Escribe en consola  
    .start()                   # Inicia la query de streaming  
)  
  
query.awaitTermination()      # Espera hasta que se detenga la query
```



• Entradas de prueba enviadas por netcat

- Eventos con timestamp, equipo, tipo de evento y xG
- Se define un esquema explícito para parsear el JSON
- El campo ts permite aplicar ventanas temporales y watermark

Terminal Shell Terminal

```
{"ts": "2025-06-01 10:00:10", "team": "RMA", "event": "goal", "xg": 0.9}  
{"ts": "2025-06-01 10:00:40", "team": "FCB", "event": "shot", "xg": 0.2}  
{"ts": "2025-06-01 10:02:55", "team": "RMA", "event": "shot", "xg": 0.3}  
{"ts": "2025-06-01 09:45:40", "team": "FCB", "event": "shot", "xg": 0.2}
```



Python Python Code

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import (
    StructType, StructField,
    StringType, DoubleType, TimestampType
)

spark = (
    SparkSession.builder
    .appName("FootballStreamingGoalEvents")
    .getOrCreate()
)
```



- **Definir el esquema explícito del evento JSON**

- Cada línea del socket es un objeto JSON con 4 campos

Python Python Code

```
schema = StructType([
    StructField("ts",      TimestampType()), # Timestamp del evento
    StructField("team",    StringType()),     # Equipo que realizó la acción
    StructField("event",   StringType()),     # Tipo de evento (goal, shot, etc.)
    StructField("xg",      DoubleType()),     # Goles esperados del evento
])
```



- **Leer y parsear eventos JSON desde el socket**
 - Cada línea recibida se parsea con el esquema definido

Python Python Code

```
events = (  
    spark.readStream  
        .format("socket")  
        .option("host", "localhost")  
        .option("port", 9999)  
        .load() # Columna "value" con el JSON crudo  
        .select(F.from_json("value", schema).alias("e")) # Parsea JSON con el schema  
        .select("e.*") # Desempaqueta los campos  
)
```



- **Aplicar watermark y agregar en ventanas**

- Tomamos ventanas de 1 minuto a modo de ejemplo
- El watermark descarta eventos que llegan con más de 2 minutos de retraso
- La ventana fija agrupa en bloques de 1 minuto según el timestamp del evento

Python Python Code

```
counts = (  
    events  
    .filter(F.col("event").isin("goal", "shot")) # Filtra solo acciones de gol  
    .withWatermark("ts", "5 minutes")           # Descarta eventos muy tardíos  
    .groupBy(F.window("ts", "1 minute"), "team") # Ventana de 1 min por equipo  
    .agg(F.count("*").alias("total"))           # Cuenta acciones por ventana  
)
```



- **Escribir resultados en consola con trigger periódico**

- El trigger actualiza los resultados cada 10 segundos

Python Python Code

```
query = (  
    counts.writeStream(  
        .outputMode("update")           # Emite resultados actualizados  
        .format("console")  
        .trigger(processingTime="10 seconds") # Ejecuta un micro-batch cada 10 s  
        .start()  
    )  
  
    query.awaitTermination()
```



- Enviar eventos con timestamps desordenados

Terminal Shell Terminal

Batch 1 de eventos

```
{"ts": "2025-06-01 10:00:10", "team": "RMA", "event": "goal", "xg": 0.9}  
{"ts": "2025-06-01 10:00:40", "team": "FCB", "event": "shot", "xg": 0.2}  
{"ts": "2025-06-01 10:02:55", "team": "RMA", "event": "shot", "xg": 0.3}
```

Terminal Shell Terminal

Evento tardío (llega después pero ocurrió antes)

```
{"ts": "2025-06-01 09:45:40", "team": "FCB", "event": "shot", "xg": 0.2}
```



- **Spark Structured Streaming para análisis en tiempo real**
 - Unifica batch y streaming con un modelo declarativo
 - Actualiza resultados incrementalmente y gestiona estado, tardíos
- **Checklist práctico de desarrollo**
 - Establecer fuente y definir el esquema de eventos
 - Identificar tiempo de evento y configurar watermarks/ventanas
 - Escoger destino y modo de escritura según necesidad
 - Definir trigger para controlar frescura de resultados
 - Ejecutar `start()` y `awaitTermination()`



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>